

Project Datavisualisatie

Dataset kiezen

We bekeken enkele datasets op de websites van StatBel en de Vlaamse overheid omdat we wel graag iets wouden onderzoeken in onze eigen leefomgeving. Daar vonden we de gebruikte datasets onder de titel “Focus op de arbeidsmarkt”.

<https://data.gov.be/nl/datasets/dda0fac53f00e067fb071ebdd70986dfc6ce691a>

Aangezien we in onze masterjaren zitten en al door andere vakken worden klaargestoomd voor de arbeidsmarkt, had het onze aandacht gekregen.

Dataset analyseren

De dataset bestaat uit veel verschillende Excel-bestanden. Deze focussen zich op ondervragingen van jongeren in de arbeidsmarkt over de overeenkomst van hun job met hun opleiding en vaardigheden alsook mogelijke probleempunten zoals ervaring van stress, arbeidsongevallen etc.

We besloten ons te focussen op de data die zich focust op overeenkomst van een job met de opleiding en vaardigheden van jongeren.

In het bestand over vaardigheden vonden we dat er een vaardigheid ‘CALCULATE’ bestudeerd was. Als wiskundigen, trok dit uiteraard de aandacht. In de pagina hierover zagen we dat deze vaardigheid vergeleken werd met alle anderen, zo ontstond een idee om te checken hoeveel overeenkomsten er zijn zodanig dat we weten welke vaardigheden vaak verwacht worden voor jobs die berekeningen moeten maken. We besloten een soort ‘Heatmaps’ te maken die aangeven hoe de populatie verdeeld is over de niveaus van een vaardigheid en de niveaus van de vaardigheid ‘CALCULATE’.

In het bestand over opleidingsniveau vonden we vergelijkingen voor verscheidene groepen in vergelijkbare categorieën zoals bv. man/vrouw, Vlaanderen/Wallonie/Brussel, etc. Zo kwamen we tot een optie om een onderzoekende functie te maken waarbij de gebruiker van de ene groep naar de andere kan overschakelen om te zien hoe groot het verschil is in deze groepen per categorie.

Eerste verkenning

Midden maart (ongeveer van 16-20 maart) hebben we de github geopend en het project opgestart. Daarnaast werden er in Observable Notebook al wat bekeken hoe de gegeven tools werken.

Er werd gebotst op een klein probleem in de gekozen dataset. In plaats van een CSV of een enkelvoudige Excel-sheet, bestond deze dataset uit Excelbestanden die duidelijk werden opgesteld met als bedoeling om gelezen te worden, en niet verwerkt. De data stond in verschillende opgemaakte tabelletjes onder elkaar. Dit maakte het onmogelijk om simpelweg om te zetten naar een CSV via een omzetprogramma.

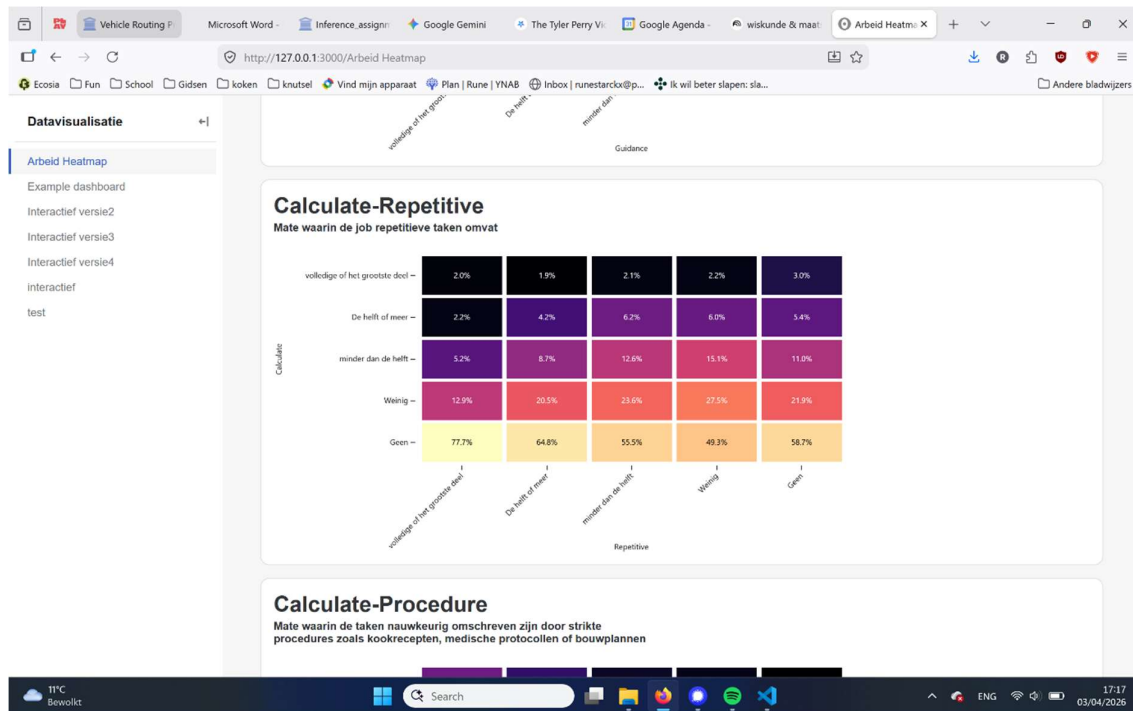

```
function makedata(beginrij){
  const array = [[0,0,0,0,0], [0,0,0,0,0], [0,0,0,0,0], [0,0,0,0,0],[0,0,0,0,0]];
  {
    for (let i = 0; i < 5; i++) {
      array[i][0] = data[beginrij+i].E
    }
    for (let i = 0; i < 5; i++) {
      array[i][1] = data[beginrij+i].G
    }
    for (let i = 0; i < 5; i++) {
      array[i][2] = data[beginrij+i].I
    }
    for (let i = 0; i < 5; i++) {
      array[i][3] = data[beginrij+i].K
    }
    for (let i = 0; i < 5; i++) {
      array[i][4] = data[beginrij+i].M
    }
  }
  // 2. Data vlakmaken voor Plot
  const dataplat = array.flatMap((row, i) =>
    row.map((value, j) => ({
      rij: niveaus.get(i),
      kolom: niveaus.get(j),
      waarde: value
    })))
  );
  return dataplat;
}
```

Na wat verkenning werd op 23 maart het Observable Framework toegevoegd aan de github. Aan de hand van de tutorials voor het Observable Framework kon er met Plot.plot() een eerste versie van een heatmap gemaakt worden. Dit werd besproken op de eerste feedback op 23 maart.

Heatmaps Vaardigheden verbeteren

We besloten dus om een analyse te maken van de data over vaardigheden rond Calculate. Na de initiële Heatmap gemaakt was, was het simpel genoeg om er voor alle vaardigheden een te maken. Deze werden dan bekeken om de interessantste te kunnen vergelijken.

De eerste versie nam de procentuele data van de tabellen, deze gaven aan wat het percent was binnen dat niveau van calculate, maar zo sommeerde de hele grafiek dus niet echt tot 1, wat stoorde. Dit werd later aangepast.



Rond 30 maart waren we tevreden met hoe deze er al uitzagen initieel en schoven we dus even de focus naar de interactieve pagina. We wouden eerst een initiële versie van alles om dan later meer details af te werken.

Toevoeging interactieve pagina

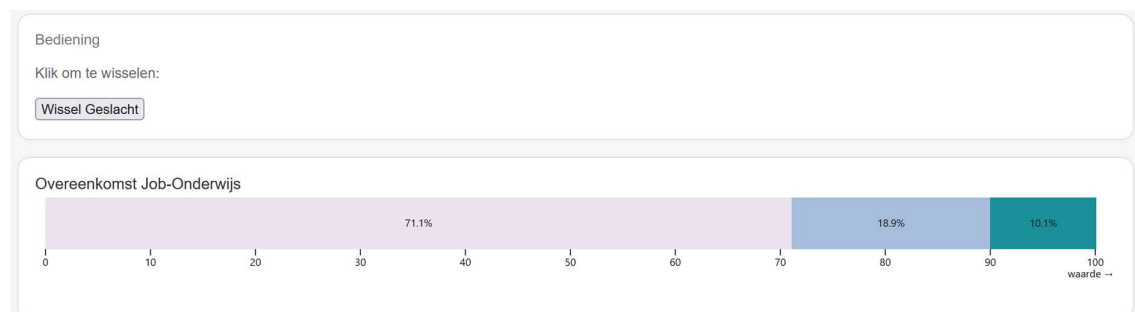
Voor een normalised horizontale bar-chart werden initieel ook de `Plot.plot()` tutorials van Observable gebruikt. Dit werkte prima voor de statische bar-charts. Voor de interactie wouden we echter dat deze bars zich zouden veranderen. We testten dus het toevoegen van een knop om te switchen tussen de data over mannen en vrouwen. Door een simpele knopwaarde, een map met (data voor mannen, 0), (data voor vrouwen, 1) en de een plot die als data de knopwaarde $\% 2$ uit de map haalde, lukte het snel om een

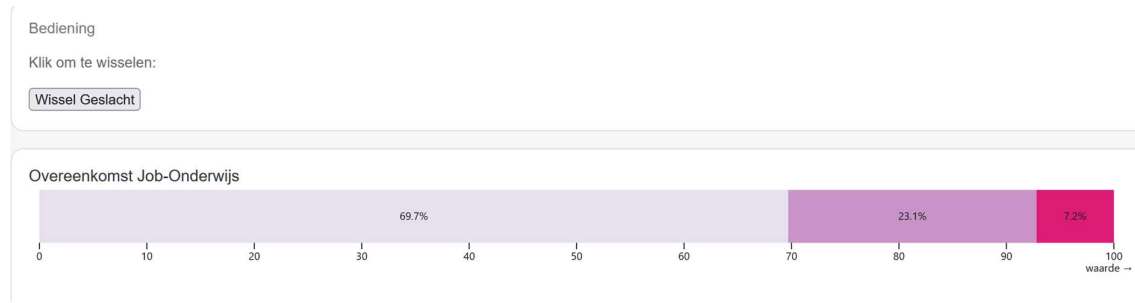
wisslende chart te maken.

```
function geslachtPlot(width){
return Plot.plot({
  key: "geslacht-plot",
  width: width,
  marginLeft: 30,
  height: 60,
  x: { domain: [0, 100], label: "Percentage (%)" },
  color: {
    scheme: clicks % 2 === 0 ? "pubugn" : "purd",
    domain: Array.from(labels1.values()),
    legend: true
  },
},
y:{axis:null},
marks: [
  Plot.barX(genderMap.get(clicks % 2), Plot.stackX({
    x: "waarde",
    y: "groep",
    fill: "categorie",
    inset: 0.5,
    key: "categorie",
    transition: { duration: 1000, easing: "cubic-in-out" }
  })),
  Plot.textX(genderMap.get(clicks % 2), Plot.stackX({
    x: "waarde",
    text: d => `${d.waarde}%`,
    key: "categorie",
    transition: { duration: 1000 }
  })))
]
})
```

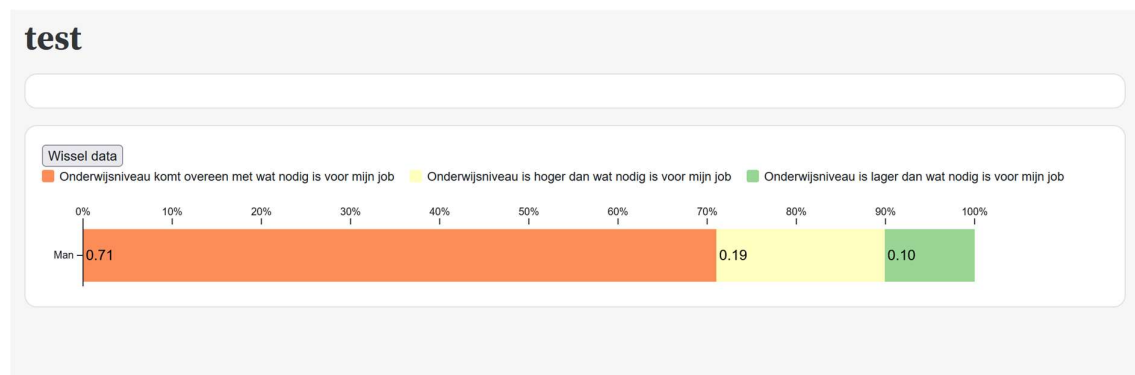
Er waren 2 grote problemen met deze aanpak; Als eerste was het een beetje slordig en niet ideaal aangezien veel maps gemaakt zouden moeten worden. Ten tweede leek het 'transition' attribuut compleet genegeerd te worden.

We wisten van de voorbeelden in de d3 Observable notebooks dat het mogelijk moest zijn om de grafieken met een animatie te veranderen, dus dat was wel het doel.





Op 1 april (geen grap) vonden we na wat getest met namaak-data in een aparte testpagina dat het lukte om een grafiek de gewenste animaties te geven, door gebruik te maken van d3 en een aparte functie die de balken zelf veranderd en dan een transform met gegeven duration gebruikt. Het leek erop dat de Observable bij een Plot.plot() gewoon de grafiek opnieuw laad als er iets veranderd in javascript in plaats van het transition-attribuut te gebruiken. Het gebruik van d3 en een knop met een listener-event die een update functie oproept die de d3 onderdelen aanpast, omzeilde dat.



Met deze nieuwe aanpak werden dus functies gemaakt die de nodige bar-chart aanmaakte alsook zijn bijhorende aparte update-functie.

Bij het browser voor oplossingen, vonden we ook de html-objecten selection. Dit sloot mooi aan bij het plan voor deze grafieken. De input van een selection kon namelijk ook de naam zijn van het stukje data dat we wouden gebruiken. Er werd nu een filter-functie gebruikt die de selection input kon gebruiken om te kiezen welk deel van de obgestelde dataset gebruikt moest worden.

Dit samen met de functions die we nu definieerden werden op 2 april gebruikt om voor de verschillende categorieën een vergelijkende bar toe te voegen.

Dezelfde dag besloten we ook dat we deze niet enkel allemaal apart zouden willen, maar dat het voor vergelijken ook goed zou zijn om ze gestapeld te hebben. Er moest even gesleuteld worden maar met de Observable tutorial voor normalised horizontal stacked bar chart, lukte dit uiteindelijk ook.

<https://observablehq.com/@d3/stacked-normalized-horizontal-bar/2>

In de week erna werden kleine aanpassingen gedaan; teksten draaien, margins en padding aanpassen, kleuren kiezen etc. Deze werden gecommitt op 16 april.

Aanpassingen heatmaps

Voor de feedback op 20 maart herbekeken we ook nog eens onze analyse van vaardigheden. We pasten zoals hierboven gezegd de percentages zodat ze aangaven wat het aandeel van de populatie in dat stukje was. We besloten dat het hierdoor wel wat lastiger was om de groep waar wij het meest in geïnteresseerd waren te beschouwen, namelijk de groep die veel berekeningen maakt. We besloten om een grafiek toe te voegen om deze specifieke groep onder de loep te nemen. We schreven onze ondervindingen in teksten bij de meest interessante grafieken. We kozen voor een grafiek met een hoge positieve correlatie, een hoge negatieve correlatie en dan eentje die redelijk even was voor alle niveaus van de 'CALCULATE' groep.

Feedback implementeren

Op 20 maart kregen we feedback op ons project. Enkele aanpassingen werden daarna gemaakt. De kleuren werden aangepast, de teksten werden verplaatst, de interactieve plots kregen de optie om te verdwijnen en voor leesbaarheid werden tooltips toegevoegd die extra uitleg konden geven.

Takenverdeling

Rune voorzag de implementaties van de interactieve datavisualisaties, alsook het werken met de github en Observable.

Astrid analyseerde de data en schreef de teksten.

Samen bekeken we de geanalyseerde heatmaps en bespraken we nodige aanpassingen en feedback.